

Анализ логики редактирования задолженности и пересчёта динамики

1. Редактирование суммы долга (FinanceQuickEditAccountView и FinanceViewModel)

В **FinanceQuickEditAccountView** реализован ввод новой суммы и сохранение через метод `saveAmount()`. Поле ввода запрещает отрицательные значения (`isValidAmount` возвращает `false` при `amount < 0`), поэтому пользователь не сможет напрямую ввести отрицательную задолженность (например, переплату по кредиту). Это предотвращает ввод некорректных данных, однако стоит учесть случай переплаты: сейчас переплаченная задолженность отображается как 0 (т.к. отрицательные значения обнуляются) – возможно, имеет смысл показать **отрицательную задолженность** как отдельный случай (чтобы пользователь видел, что кредит переплачен).

При сохранении вызывается действие `viewModel.handle(.updateAccountAmount(account, amount))` ¹. В обработке **FinanceViewModel.updateAccountAmount** логика различается для типов счетов:

- **Кредитная карта** (`account.accountType == .card` с `card.cardType == .credit`): Метод вычисляет **старую задолженность** `oldAmount` как остаток лимита `limit - oldBalance` (где `oldBalance = card.balance`) ². Затем устанавливает **новое значение баланса карты** `card.balance` так, чтобы получилась заданная задолженность: `newBalance = max(0, limit - newDebt)` ³. Это означает, что если пользователь вводит задолженность больше кредитного лимита, баланс установится в 0 (долг ограничен лимитом). Возможно, стоит обработать этот сценарий отдельно (например, сообщать пользователю об ограничении лимита вместо молчаливого округления до лимита).
- **Кредит (заём)** (`account.accountType == .credit`): Метод напрямую обновляет `credit.remainingAmount = newAmount` ⁴ без каких-либо ограничений кроме неотрицательности. Здесь важно, что вводится **общий остаток долга** кредита (а не сумма платежа и т.п.).

После установки новых значений метод сохраняет изменения в контексте и **создаёт транзакцию типа** `.balanceAdjustment` (ручная корректировка баланса) для фиксации изменения долга/баланса. В эту транзакцию записывается величина изменения `difference = newAmount - oldAmount`. В коде есть пояснения по знакам для разных случаев:

- Для карт: «положительное значение = увеличение баланса (income), отрицательное = уменьшение (expense)» ⁵. То есть, если `difference` положительный, это трактуется как поступление средств (рост баланса карты), если отрицательный – как расход (уменьшение баланса). Эта логика верна для дебетовых карт (рост баланса = доход, уменьшение = расход). Однако для кредитных карт смысл обратный: увеличение задолженности

означает, что баланс карты фактически **уменьшился** (вы потратили средства). В текущей реализации код не делает различий – он использует тот же `difference` и знак, что может привести к неправильной интерпретации транзакции для кредитки. *Возможное улучшение:* для кредитных карт инвертировать знак `difference` при создании транзакции. Например, вычислять изменение как разницу баланса: `balanceDiff = newBalance - oldBalance`. Тогда для кредитки положительный `balanceDiff` означал бы реальный рост доступного баланса (погашение долга), а отрицательный – уменьшение баланса (рост долга). Это обеспечит правильную трактовку в динамике (см. раздел 3 ниже).

- Для **кредитов (заёмов)**: «положительное значение = увеличение остатка долга (*expense*), отрицательное = уменьшение (*income*)»⁶. Т.е. пост `remainingAmount` рассматривается как расход (долг вырос), а снижение долга – как «доход» (с точки зрения улучшения финансового состояния). Здесь знак `difference` уже соответствует этой трактовке: если пользователь увеличил долг, `difference` будет положительным, если погасил часть – отрицательным. Код сохраняет этот знак как есть.

Важно отметить, что **FinanceQuickEditAccountView** не допускает ввода отрицательных сумм, поэтому сценарий уменьшения долга ниже нуля исключён. Но если задолженность кредита или карты была изначально отрицательной (переплата), UI сейчас покажет 0 из-за логики `max(0, ...)`. Это место кода можно доработать для более корректного отображения переплаты.

2. Обновление данных после редактирования (FinanceViewModel и FinanceDynamicsViewModel)

После изменения суммы долга **FinanceViewModel.updateAccountAmount** обновляет основные данные приложения:

- Вызывается `loadAccounts()` и пересчитывается общая сумма всех счетов `calculateTotalAmount()`⁷⁸. Это обновит списки счетов (включая изменённый баланс карты или остаток кредита) и общий итог. **Групповые суммы** также пересчитываются: в коде для кредиток и кредитов после сохранения есть блок, который находит группу, к которой принадлежит счёт, и асинхронно вычисляет её новую сумму с помощью `calculateGroupTotal`⁹¹⁰, затем обновляет словарь `state.groupTotals`.

- **Потенциальные проблемы обновления в FinanceViewModel**: пересчёт группы выполняется внутри `Task` без ожидания завершения. В результате UI может на мгновение отображать старую сумму группы до тех пор, пока задача не завершит вычисление и не обновит состояние. Желательно либо ожидать этот расчёт перед закрытием окна редактирования, либо явно обновлять UI (например, через временное состояние "обновляется..."). Кроме того, если счёт не принадлежит ни одной группе (непривязанный), то пересчитывается только общий итог (группы в этом случае не задействованы). В целом, после редактирования метод **не вызывает** `loadGroups()`, поэтому структура групп остаётся прежней (группа, счета внутри и т.д. не меняются). Это нормально, так как мы изменили только сумму, но нужно учитывать, что объекты группы и счета – это SwiftData-модели. Их поля (`balance`, `remainingAmount`) обновились в контексте и SwiftUI, вероятно, отобразит новые значения автоматически. Если же где-то в UI использовались закэшированные данные, может потребоваться вручную оповестить об

изменении. В предоставленном коде основная привязка идёт к самим объектам модели, поэтому после `try modelContext.save()` изменения должны подтянуться в SwiftUI (при условии, что `SwiftData/ObservableObject` отслеживает эти поля).

- **FinanceDynamicsViewModel** (экран “Динамика”): чтобы отразить изменения, нужно заново загрузить данные. Код **Dynamics**-экрана показывает, что при появлении экрана вызывается `dynamicsViewModel.handle(.loadData)`¹¹. Этот метод вытягивает актуальные данные из **FinanceViewModel** и базы данных: обновляет список групп, заново загружает все карты, кредиты, инвестиции из базы, и **получает все транзакции** типа `CashflowTransaction`¹²¹³. Таким образом, если открыть экран *Динамики* после редактирования долга, пользователь увидит обновлённую картину.

Однако, если экран *Динамики* уже был открыт, есть риск, что он не узнает об обновлениях автоматически. В коде `FinanceDynamicsView` есть привязка `.onChange` для количества карт¹⁴ – при изменении количества карт динамика перезагрузится. Но аналогичного наблюдателя для кредитов нет. То есть, если пользователь отредактировал существующий кредит (не добавил/удалил, а именно обновил сумму долга), свойство `availableCredits.count` не изменится, и автоматического вызова `.loadData` не произойдёт. *Возможное решение:* добавить наблюдение за изменением, например, `financeViewModel.state.availableCredits` (количества или даже конкретных полей, если возможно) и вызывать `dynamicsViewModel.loadData()` при обновлении. Либо отправлять явный сигнал/событие в динамику после редактирования. В текущем коде есть обработка `.onAppear` (повторно загружает данные при возвращении на экран)¹⁵, поэтому если пользователь закрыл и вновь открыл экран динамики – данные обновятся. Но **в реальном времени на уже открытом экране** изменения кредита могут не отобразиться без этих дополнительных мер.

- **Кэширование значений:** `FinanceDynamicsViewModel` активно использует кэши (`cardsCache`, `creditsCache`, `transactionsByCardCache` и т.д.) для оптимизации расчетов¹⁶. Каждый вызов `loadData()` сбрасывает эти кэши через `rebuildCaches()`¹³¹⁷ и заново заполняет. Это важно, чтобы учесть свежие данные. Возможная проблема – если изменения произошли, а `loadData()` не вызывался, динамика может продолжать использовать устаревшие данные из кэша. Например, `initialBalancesCache` хранит начальный баланс карт при создании, `transactionsBy...Cache` – список транзакций по счетам. После ручной правки долга появляется новая транзакция `.balanceAdjustment`, которая в кэше не учтена до перезагрузки. Поэтому ещё раз акцент: необходимо вызывать `loadData()` при каждом изменении данных, либо обеспечивать инвалидацию соответствующих кэшей. В данном коде это делается при открытии/возврате на экран, но **не в момент сохранения долга**. Для надёжности можно вызвать что-то вроде `financeDynamicsViewModel.handle(.loadData)` прямо в `updateAccountAmount` (если `dynamics-VM` есть и активна) или послать `Notification`/флаг, на который `DynamicsViewModel` подпишется.

3. Пересчёт динамики (FinanceDynamicsViewModel)

На экране “Динамика” рассчитываются показатели изменения баланса счетов/задолженности за выбранный период. Рассмотрим ключевые аспекты: **текущий баланс**, **процент погашения (дельта в %)** и **график задолженности**, а также возможные ошибки в расчётах.

• **Текущий баланс (currentBalance):** В state есть поле `currentBalance`, которое обновляется в функции `updateCurrentBalanceAndDelta()`¹⁸. Этот метод суммирует балансы всех выбранных счетов на определённую дату (либо на текущий момент, либо на выбранную точку графика). Здесь кроется нюанс: **для кредитных карт и кредитов в вычислениях балансами считаются суммы задолженности**. В коде внутри `calculateBalanceAtDate` видно, что:

- Для карт после применения всех транзакций рассчитывается `accountBalance` — если карта кредитная, то **переключают баланс в задолженность**:
`accountBalance = max(0, limit - cardBalance)`¹⁹. То есть вместо остатка на карте хранится сумма долга (всегда неотрицательная благодаря `max(0, ...)`).
- Для кредитов вызывается `calculateCreditRemainingAmount`, которая возвращает **остаток долга** на заданную дату²⁰. Этот остаток тоже всегда неотрицательный (в функции используется `max(0, ...)` при расчетах).

Таким образом, **в агрегированном режиме** динамики сумма текущего баланса `currentBalance` фактически складывает активы и задолженности как положительные числа. Например, если выбраны и счета с деньгами, и кредиты, они будут просто суммированы. Это **не нетто-баланс**, а суммарная величина (долги не вычитаются). Такое решение может быть осознанным (например, для отображения общей стоимости активов отдельно от общей задолженности), но тогда интерпретация дельты должна быть внимательной.

Проблема знака дельты: Если в выборке только долги (например, мы смотрим динамику конкретного кредита или группы кредитов), уменьшение долга приведёт к **уменьшению** `currentBalance` (потому что в расчёте это положительное число стало меньше). Соответственно, разница `delta = currentBalance - startBalance` будет **отрицательной** при погашении долга. Формула сейчас именно такая²¹: разность рассчитывается прямым вычитанием без учета типа счета. Получается, погашение кредита отображается как отрицательная дельта, рост долга – как положительная. Хотя с точки зрения «состояния финансов» это верно (суммарный баланс активов+долгов снизился, если долг уменьшился), с точки зрения **процента погашения долга** это сбивает с толку – интуитивно ожидается, что **погашение** показывается как положительный прогресс.

В коде эта нестыковка частично обработана **только в детализации (breakdown)**. В `updateDynamicsBreakdown()` при вычислении элементов для каждого счета или группы есть специальная коррекция: «Для кредитных карт и кредитов: уменьшение долга = положительная дельта (инвертируем дельту)»²². Там вводится переменная `delta`, которая для кредитов/кредитных карт берётся с обратным знаком `delta = -rawDelta`²³. Процент изменения тоже считают относительно исходного долга, после инверсии знака дельты²⁴²⁵. Это значит, что **в списке динамики для отдельных счетов/групп уменьшение задолженности отразится положительной цифрой (+X% погашено)** – что правильно.

Однако в верхней панели (`currentBalance` и `periodDelta`) такой инверсии **не выполняется**. Там дельта и процент рассчитываются напрямую по совокупному балансу ²¹. Например, если за месяц вы выплатили 10% кредита, верхний индикатор может показать “-10%” (падение суммарного баланса), тогда как в разбивке по счету будет “+10%” погашения. Это потенциально **сбивает пользователя с толку**. Рекомендуется унифицировать подход: либо в агрегированном режиме и верхней панели тоже учитывать знак для долгов, либо хотя бы явным образом показывать, что отрицательное значение означает снижение долга (например, цветом или пояснением). Один из вариантов – если выборка содержит **только долговые счета** (или вообще, для каждого кредитного счета считать дельту с инверсией, как это сделано в `breakdown`), тогда `periodDelta` можно посчитать с учётом инверсии знаков. Это потребует определять, какие счета участвуют, и, например, суммарно брать дельту со знаком, противоположным долгам.

- **Вычисление процента погашения:** Как отмечено выше, в `breakdown` для каждого кредита/кредитной карты процентная дельта рассчитывается относительно начальной задолженности. После инверсии дельты формула `deltaPercent = (delta / |startBalance|) * 100` показывает, на сколько процентов изменилась задолженность. Например, было 100 000₽, осталось 80 000₽ – долг уменьшился на 20% (будет показано +20%). Эта логика верна и полезна. В коде предусмотрены проверки на нулевой начальный баланс, чтобы избежать деления на ноль (в таких случаях выставляются условные большие значения 999999 или 0) ²⁶ ²⁷. **Граничные случаи:** если на начало периода долг был ~0, а потом появился (например, взяли новый кредит), алгоритм сейчас поставит огромный положительный процент (+999999%), хотя по смыслу появление долга – негативная динамика. И наоборот, если долг полностью погашен к концу периода, а в начале был существенным, покажется +100% или более (что в целом ок – 100% погашено). Эти крайности, вероятно, оставлены как есть для упрощения (999999% как символ “очень много”). *Возможное улучшение:* для кредита, появившегося с нуля, можно отдавать отрицательный большой процент, но это деталь UX. Главное, что **расчёт процента погашения в `breakdown` сделан правильно**, с учётом инверсии знака дельты для долгов.

- **Пересчёт данных графика:** График строится на основе исторических значений баланса/ задолженности по дням. `FinanceDynamicsViewModel` собирает все транзакции и рассчитывает баланс на каждую значимую дату. Здесь тоже важно корректно учесть знаки и типы счетов:

- Для **обычных счетов и дебетовых карт** транзакции типа `.income` добавляют к балансу, `.expense` вычитают и т.д., а `.balanceAdjustment` просто корректирует баланс на величину `transaction.amount` (положительное – повысит баланс, отрицательное – снизит) ²⁸ ²⁹. Это соответствует ожиданиям для активов.
- Для **кредитных карт** стратегия такая: сначала рассчитан *баланс карты* (как если бы это был актив), а в конце он переводится в задолженность = `limit - balance`. Ошибка кроется в применении ручных корректировок (`balanceAdjustment`) в процессе расчёта **баланса кредитной карты**. В текущем коде **нет особого случая для кредитных карт** при применении этих транзакций – используется тот же блок, что и для дебетовых карт ²⁹. Там «*положительное значение увеличивает баланс, отрицательное уменьшает*». Но для кредитки положительный `transaction.amount` означает, что мы вручную **увеличили долг** (например, транзакцией +100 мы указали, что долг стал больше на 100). В реальности увеличение долга **уменьшает баланс карты**. Код же возьмёт это +100 и **прибавит к балансу карты** ³⁰, то есть отработает как будто баланс **увеличился**. В результате смоделированный баланс на графике будет выше, чем должен, а рассчитанная

задолженность (`limit - balance`) – ниже реальной. Проще говоря, график покажет, что долг снизился или даже стал отрицательным, хотя на самом деле он вырос.

Пример ситуации: кредитный лимит 100 000₽, текущий долг 50 000₽ (баланс карты = 50 000). Пользователь вручную увеличил долг до 60 000₽. В `updateAccountAmount` записали транзакцию `.balanceAdjustment` с `amount = +10 000` ₽. При построении графика: - До корректировки баланс был 50 000, задолженность 50 000. - Транзакция +10 000₽ сохранена (долг вырос). - Алгоритм графика применяет её: **прибавляет** 10 000 к балансу карты (решив, что это как доход) – баланс становится 60 000. - Затем вычисляет задолженность = 100 000 - 60 000 = 40 000₽. Вместо реальных 60 000₽ долг показан 40 000₽, т.е. график ошибочно отобразит **снижение** задолженности.

Это серьёзная логическая ошибка. Она возникает из-за того, что знак транзакции для кредитной карты интерпретируется неправильно. *Решения:* 1. **Корректировать знак при сохранении транзакции** – как упоминалось, для кредитных карт можно сохранять `difference` с обратным знаком. Тогда, в примере выше, рост долга 10 000₽ сохранился бы как транзакция -10 000₽. При построении графика -10 000₽ **вычтется** из баланса карты, что правильно снизит баланс до 40 000 и даст задолженность 60 000₽ – корректно. 2. **Либо учитывать тип карты при расчёте** – в `calculateBalanceAtDate` в блоке `.balanceAdjustment` добавить условие: *если карта кредитная*, то вычитать сумму вместо сложения. Например:

```
case .balanceAdjustment:
  if transaction.cardID == card.cardUniqueID {
    let converted = ... // конвертация суммы
    if card.cardType == .credit {
      cardBalance -= converted // для кредитки вычитаем
    } else {
      cardBalance += converted // для дебетовой добавляем
    }
  }
```

Такой код отразит ручные изменения правильно для обоих видов карт.

Учтём, что аналогичная проблема с трактовкой транзакций может возникать и в **расчёте кредитов**. В `calculateCreditRemainingAmount` все `.balanceAdjustment` транзакции отфильтровываются и затем *вычитаются* из остатка долга³¹. Комментарий гласит: «положительное *amount (income)* уменьшает долг, отрицательное (*expense*) увеличивает»³². Предполагается, что положительная транзакция означает внесение денег (уменьшение долга), отрицательная – дополнительный расход (рост долга). Но при сохранении мы записали транзакцию с *тем же знаком, что и разница в долге*: рост долга записан как положительная сумма (хотя по смыслу это “expense”), снижение долга – как отрицательная. Получается, сейчас динамика кредита будет трактовать эти знаки **наоборот**: - Если долг был увеличен (мы сохранили +X), функция воспримет +X как “income” и **отнимет X от остатка**, ошибочно занизив долг. - Если долг уменьшен (мы сохранили -Y), функция увидит отрицательное как “expense” и **прибавит Y к остатку**, завывсив долг.

Таким образом, при ручной правке кредита график и вычисление текущего остатка могут давать неверные цифры. Решение сходно с предыдущим: нужно согласовать знак. Либо

хранить *уменьшение* долга как положительное значение, а *увеличение* долга – как отрицательное (инвертировать перед записью в транзакцию для кредитов), либо менять формулу расчёта. Более простой и безопасный путь – именно менять знак при создании транзакции для кредитов и кредитных карт, чтобы **internal logic** динамики оставалась целостной. То есть: - При увеличении долга (кредит или кредитка) записывать отрицательный `amount` в `CashflowTransaction`. - При уменьшении долга – положительный `amount`.

Тогда код динамики можно оставить как есть, и он начнёт корректно работать согласно комментариям. Если же менять динамику, то придётся в нескольких местах условно инвертировать применение транзакции.

- **Обновление графика после редактирования:** ещё один момент – когда пользователь меняет сумму долга, создаётся транзакция *в момент текущей даты*. График при следующей загрузке данных её учтёт. В идеале, чтобы график обновился **сразу**, можно после сохранения долга (если `DynamicsView` открыт) либо пересчитать график, либо добавить точку “сейчас” с новым значением. В текущем коде решено проще: повторный вызов `loadData()` пересчитывает транзакции и вызовет `updateChartData()`, график перестроится. Поэтому, главное, как отмечалось, – не забыть вызвать загрузку динамики.

Вывод: Основные участки кода, требующие доработки: - **`FinanceViewModel.updateAccountAmount`** – обработка кредитных карт: сохранить транзакцию с правильным знаком для `difference` (сейчас знак сохраняется как есть ³³, нужно инвертировать для кредиток) и, возможно, предотвратить установку задолженности сверх лимита более явным образом. Аналогично – убедиться, что для кредитов знак транзакции соответствует логике уменьшения/увеличения долга (сейчас – наоборот, требует инверсии либо в расчёте, либо при сохранении) ³⁴. - **`FinanceDynamicsViewModel.calculateBalanceAtDate`** – в секции применения `.balanceAdjustment` добавить условие для кредитных карт, чтобы корректировать баланс обратным образом ²⁹. Это устранил ошибку отображения динамики кредитных карт при ручных правках. - **`FinanceDynamicsViewModel.calculateCreditRemainingAmount`** – пересмотр применения транзакций к остатку долга ³². В текущем виде вычитание всех `transaction.amount` неверно при знаках, которые сейчас сохраняются. Должно выполняться: *если транзакция отрицательная (долг уменьшили) – вычесть отрицательное (т.е. прибавить) к долгу?* Логика запутанная. Проще, как сказано, пусть в транзакции уже будет “правильный” знак, тогда код можно оставить. Если нет – придётся тут тоже учитывать знак (например, прибавлять для положительного). - **Инверсия дельты в агрегированном режиме** – необязательная, но желательная доработка UX: в `updateCurrentBalanceAndDelta` можно анализировать, какие аккаунты выбраны. Если все – кредиты/кредитки, то дельту процентов показывать с обратным знаком (или хотя бы отметить это визуально). Либо вовсе показывать отдельный показатель “погашено X%” для долговых счетов, вместо использования общего “дельта”. - **Обновление `FinanceDynamicsViewModel` при изменениях** – рассмотреть добавление наблюдателей на изменения количества/списка кредитов и инвестиций, аналогично картам, чтобы динамика обновлялась сразу. Также можно интегрировать обновление через `EnvironmentObject/AppState`, если таковой есть, или через `Notifications`.

Суммируя, **основные ошибки** связаны со **знаками и консистентностью расчетов задолженности**: - Некорректная трактовка знака транзакций при ручных изменениях долга (для кредитов и кредитных карт) – требует унификации между моментом сохранения и моментом расчета динамики. - Кэширование и обновление – убедиться, что после изменения долга динамика получает актуальные данные (добавить вызов `loadData` или наблюдателей). -

Отображение прогресса погашения – текущая реализация может показать отрицательный процент там, где имелся в виду положительный (и наоборот). Нужно либо переворачивать знак для долгов, либо чётко объяснять эти показатели пользователю.

Источники кода:

- Обработка сохранения суммы долга в `FinanceViewModel.updateAccountAmount` (кредитные карты и кредиты) [3](#) [35](#) .
- Создание транзакции с комментариями по знакам для карт и кредитов [5](#) [6](#) .
- Расчёт баланса карты и применение ручных корректировок (Dynamics) [29](#) [19](#) .
- Расчёт остатка кредита с применением транзакций (Dynamics) [32](#) .
- Инвертирование дельты для отображения прогресса по долгам (Dynamics breakdown) [22](#) .

[1](#) FinanceQuickEditAccountView.swift

`file:///file_000000002c6871faaf8582f0be7141d6`

[2](#) [3](#) [4](#) [5](#) [6](#) [7](#) [8](#) [9](#) [10](#) [33](#) [34](#) [35](#) FinanceViewModel.swift

`file:///file_000000003d8071fab562580b22ad778f`

[11](#) [14](#) [15](#) FinanceDynamicsView.swift

`file:///file_000000003a7471faad38ca007b0ae17d`

[12](#) [13](#) [16](#) [17](#) [18](#) [19](#) [20](#) [21](#) [22](#) [23](#) [24](#) [25](#) [26](#) [27](#) [28](#) [29](#) [30](#) [31](#) [32](#) FinanceDynamicsViewModel.swift

`file:///file_00000000fca871faa5ce7c10c98cec68`